Dagster: Modern Data Orchestrator Platform

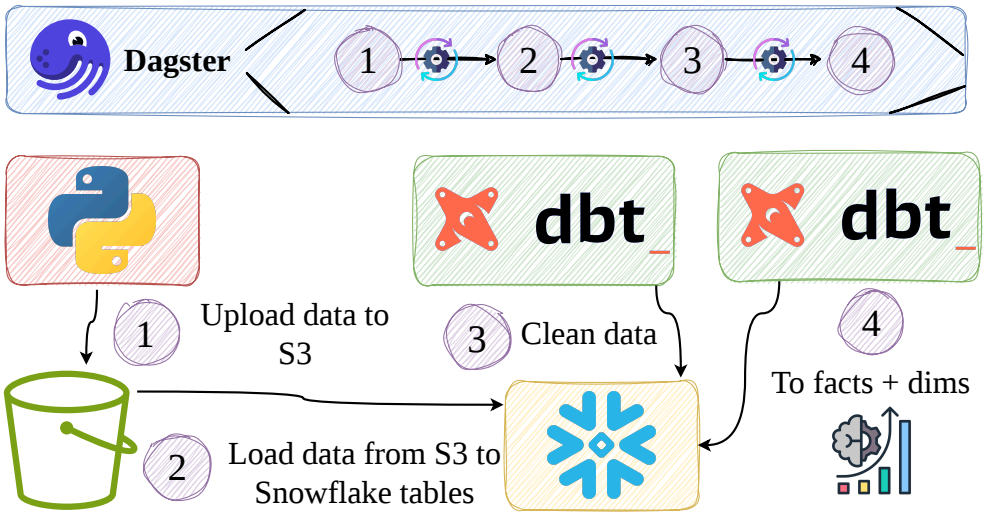
Tạ Hữu Anh Dương **Dương Nguyễn Thuận**

Proofreading: Đinh Quang Vinh

I. Bối cảnh

Một hệ thống dữ liệu thực tế hiếm khi được xây dựng trên một công cụ duy nhất. Dữ liệu thô có thể được thu thập bằng script Python, lưu trữ trên object storage như Amazon S3, nạp vào kho dữ liệu như Snowflake, rồi được làm sạch và biến đổi thành các bảng phân tích bằng dbt. Mỗi công cụ đảm nhận tốt phần việc của mình, nhưng bản thân chúng không biết gì về nhau và cũng không biết mình đang nằm ở đâu trong quy trình chung. Khi số lượng bước xử lý tăng lên, thách thức không còn nằm ở việc viết đúng từng bước, mà ở việc quản lý quan hệ phụ thuộc giữa chúng và phát hiện sự cố kịp thời.

Các công cụ điều phối được sử dụng để giải quyết vấn đề này. Nhiệm vụ của orchestrator không phải là xử lý dữ liệu, mà là quyết định bước nào chạy trước, bước nào chạy sau, xử lý khi có lỗi và ghi lại lịch sử thực thi để giám sát.



Hình 1: Vai trò điều phối của Dagster trong một pipeline dữ liệu điển hình. Dagster không trực tiếp xử lý dữ liệu mà điều phối các công cụ chuyên biệt. Cụ thể, python script đưa dữ liệu thô lên S3, dữ liệu được nạp vào Snowflake, và sau đó dbt làm sạch và biến đổi thành các bảng fact và dimension phục vụ phân tích.

Dagster tiếp cận bài toán này theo một góc nhìn khác so với các orchestrator truyền thống như Apache Airflow. Airflow tổ chức pipeline theo tác vụ, tức là người dùng khai báo những việc cần làm và thứ tự thực hiện chúng. Dagster tổ chức pipeline theo dữ liệu thông qua khái niệm Software-Defined Asset, tức là người dùng khai báo những tập dữ liệu cần có và cách tạo ra chúng, còn thứ tự thực thi được suy ra từ các mối quan hệ phụ thuộc giữa chúng. Nhờ đó, người vận hành biết được bảng nào đang lỗi thời, bảng nào chưa từng được tạo và bảng nào được tạo từ nguồn nào. Tài liệu này trình bày kiến trúc cùng các khái niệm cốt lõi của Dagster, sau đó áp dụng vào một dự án demo đơn giản trong phần thực hành.

Bảng Giải Thích Thuật Ngữ

Thuật Ngữ	Giải thích
Asset	Một sản phẩm dữ liệu có ý nghĩa được pipeline tạo ra hoặc duy trì, chẳng hạn như bảng dữ liệu, file, dataset hoặc một tập dữ liệu đã được biến đổi.
Orchestrate	Quá trình điều phối và quản lý việc thực thi các tác vụ theo trình tự và logic xác định, bảo đảm chúng phối hợp đúng cách trong một hệ thống.
Task Dependencies	Quan hệ ràng buộc giữa các tác vụ trong hệ thống, quy định tác vụ nào phải hoàn thành trước khi tác vụ khác được chạy.
Scheduler	Thành phần chịu trách nhiệm lên lịch, khởi chạy và giám sát quá trình thực thi các tác vụ theo thời gian hoặc điều kiện xác định.
Workflow	Tập hợp các tác vụ được sắp xếp có cấu trúc và liên kết logic với nhau để hoàn thành một quy trình xử lý dữ liệu hoặc nghiệp vụ.
Backfill	Quá trình chạy lại một pipeline hoặc workflow cho các khoảng thời gian trong quá khứ để xử lý dữ liệu lịch sử chưa được xử lý hoặc cần được tái xử lý.
Lineage	Thông tin mô tả nguồn gốc, quá trình biến đổi và các quan hệ phụ thuộc của dữ liệu, cho biết dữ liệu được tạo ra từ những asset nào và ảnh hưởng đến những asset nào.
Asset Graph	Đồ thị biểu diễn quan hệ phụ thuộc giữa các data asset, cho phép hệ thống xác định thứ tự và phạm vi thực thi dựa trên các dependency được khai báo.

Mục lục

I. Bối cảnh **1**

II. Kiến trúc tổng quan của Dagster **4**

II.1. Long-running Services 4

II.2. Code location server & Dagster storage 5

II.3. Quy trình thực thi tác vụ 7

III. Các khái niệm cốt lõi trong Dagster **7**

III.1. Software-Defined Assets (SDAs) 7

III.2. Ops, Asset jobs, I/O Manager, Partitions 9

III.2.1. Ops 9

III.2.2. Asset jobs 10

III.2.3. I/O Manager 10

III.2.4. Partitions 12

III.3. Schedules và Sensors 13

III.3.1. Schedules 13

III.3.2. Sensors 14

IV. Thực hành **16**

IV.1. Kiến trúc tổng quan 16

IV.2. Vai trò từng thành phần 16

IV.3. Cấu trúc thư mục 17

V. Triển khai **18**

V.1. Cài đặt và chạy 18

V.2. Chạy với cấu hình tùy chỉnh 18

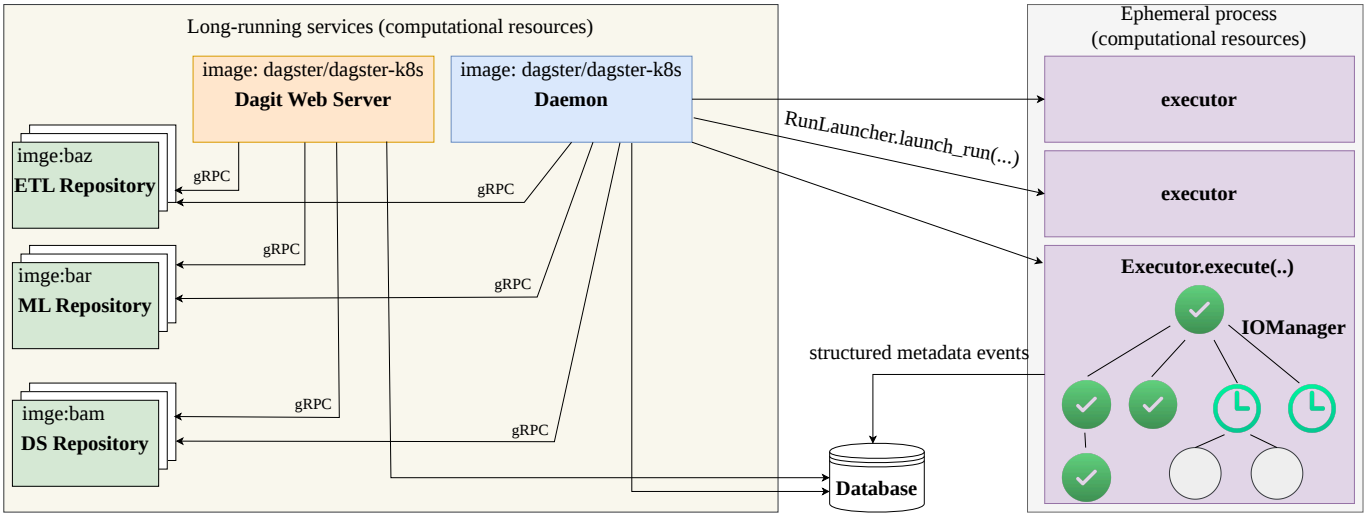
V.3. Kiểm tra dữ liệu sau khi chạy 19

V.4. Kiểm thử 19

VI. Tài liệu tham khảo **20**

II. Kiến trúc tổng quan của Dagster

Điểm nổi bật trong kiến trúc của Dagster là sự phân tách rõ ràng giữa môi trường quản lý điều phối và môi trường thực thi mã nguồn. Cách tiếp cận này giúp nâng cao tính linh hoạt, khả năng mở rộng quy mô, cũng như mức độ an toàn của toàn bộ hệ thống lưu trữ và tính toán. Phần dưới đây trình bày chi tiết các thành phần cốt lõi trong kiến trúc triển khai mã nguồn mở (Open Source Software - OSS) của Dagster.



Hình 2: Kiến trúc triển khai mã nguồn mở của Dagster. Các dịch vụ chạy thường trực gồm webserver và daemon giao tiếp với các code location thông qua gRPC, trong khi mỗi lần chạy pipeline được thực thi trong một tiến trình riêng. Toàn bộ metadata và các sự kiện thực thi được lưu vào cơ sở dữ liệu dùng chung.

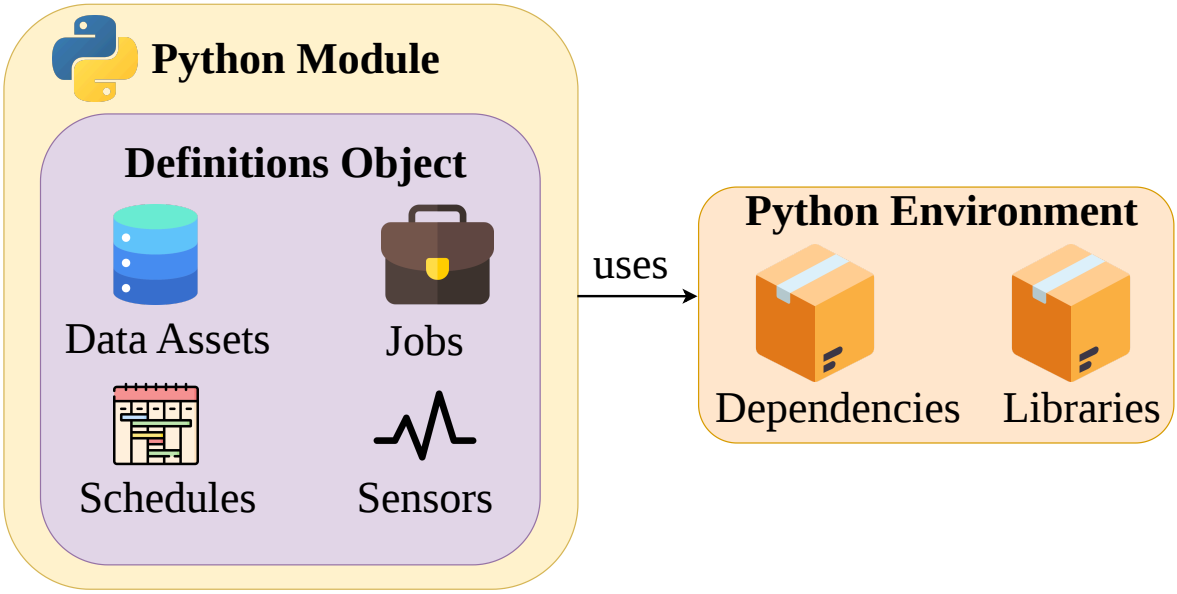
II.1. Long-running Services

Hệ thống chạy hai dịch vụ thường trực (webserver + daemon)... Ngoài ra, mã nguồn được nạp qua các Code Location Server (gRPC) do webserver/daemon khởi chạy. Mỗi dịch vụ đảm nhận một vai trò hoàn toàn độc lập:

- **Dagster Webserver:** Dịch vụ này chịu trách nhiệm cung cấp giao diện người dùng và phản hồi các truy vấn GraphQL từ các máy khách. Để đáp ứng nhu cầu truy cập lớn từ nhiều nhóm người dùng, Webserver có thể được triển khai với một hoặc nhiều replicas hoạt động song song.
- **Dagster Daemon:** Đây là trung tâm điều phối tự động của hệ thống. Daemon đảm nhiệm việc vận hành các lịch trình, và quản lý hàng đợi thực thi tác vụ.
- **Code Location Server:** Dịch vụ này có chức năng cung cấp metadata về tập hợp các assets do người dùng lập trình. Một hệ thống có thể tồn tại nhiều Code Location Server khác nhau để phục vụ nhiều luồng quy trình, nhưng mỗi Code Location chỉ được phép duy trì một bản sao duy nhất cho máy chủ của nó.

II.2. Code location server & Dagster storage

Một trong những điểm nổi bật về kiến trúc của Dagster là sự xuất hiện của khái niệm "Code Location". Code Location đóng vai trò là một điểm tham chiếu đến một module Python cụ thể chứa **Definitions** object. Object này đóng gói toàn bộ assets, jobs và cấu hình liên quan (được trực quan hóa qua hình 3).



Hình 3: Các thành phần của một Code Location: đối tượng `Definitions` nằm trong module Python và chạy trên môi trường riêng.

Đoạn code dưới đây minh họa một Code Location đơn giản: asset `my_asset` được định nghĩa bằng decorator `@asset` và đăng ký vào `Definitions` object, nhờ đó Dagster có thể nhận diện và quản lý asset này khi nạp module.

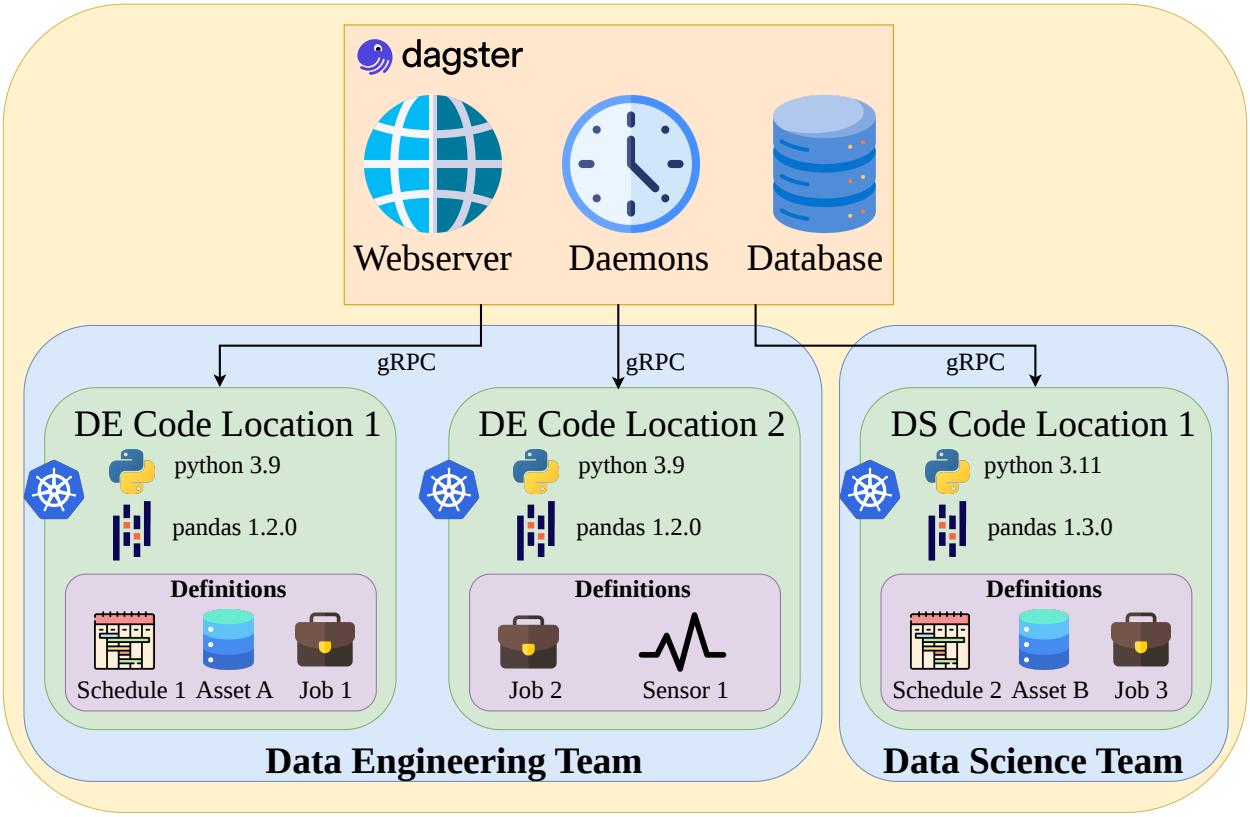
Ví dụ về Definitions object

```python
from dagster import Definitions, asset

@asset
def my_asset():
    return "Hello, Dagster!"

defs = Definitions(assets=[my_asset])
```

Một hệ thống Dagster thường không chỉ có một Code Location duy nhất. Hình 4 minh họa một kịch bản điển hình: các nhóm Data Engineering và Data Science mỗi bên quản lý các Code Location riêng. Mỗi Code Location chạy trong một tiến trình hoặc container độc lập, có môi trường Python riêng.



Hình 4: Kiến trúc nhiều Code Location: các thành phần lõi của Dagster giao tiếp qua gRPC với từng Code Location, mỗi Code Location có môi trường Python và phiên bản thư viện riêng, được quản lý bởi các nhóm khác nhau.

Cách tổ chức này mang lại một số lợi ích thực tiễn quan trọng:

- **Quản lý phụ thuộc và cô lập:** Mỗi Code Location duy trì một hệ sinh thái thư viện độc lập. Các nhóm làm việc khác nhau có thể sử dụng các phiên bản công cụ khác nhau (ví dụ: nhóm Data Engineering dùng Pandas 1.2.0, nhóm Data Science dùng Pandas 1.3.0) trong cùng một không gian máy chủ mà không xảy ra xung đột hệ thống.
- **Khả năng chịu lỗi và độ tin cậy:** Vì Code Location chạy tách biệt với các thành phần lõi của Dagster, lỗi trong mã nguồn của một Code Location (chẳng hạn lỗi import hoặc tràn bộ nhớ) chỉ ảnh hưởng đến chính nó. Webserver, daemons, schedules và sensors của các Code Location khác vẫn tiếp tục hoạt động bình thường.
- **Sự rõ ràng trong tổ chức:** Hệ thống cung cấp bộ khung phân nhóm logic với các namespaces độc lập. Thiết kế này ngăn chặn tình trạng trùng lặp tên gọi giữa các bộ phận.

Ngoài Code Location server, Dagster storage cũng là một thành phần quan trọng trong kiến trúc. Đây là nơi Dagster lưu trữ toàn bộ lịch sử vận hành của hệ thống, bao gồm metadata của các lần chạy, event log phát sinh trong quá trình thực thi, lịch sử ticks của schedule và sensor, cùng nhiều thông tin khác. Nhờ đó, giao diện web có thể hiển thị lại trạng thái của từng asset và từng lần chạy, kể cả sau khi tiến trình thực thi đã kết thúc.

II.3. Quy trình thực thi tác vụ

Quá trình khởi chạy và hoàn thành một tác vụ trong Dagster phải đi qua nhiều lớp xử lý. Thiết kế này đảm bảo logic khởi chạy và quá trình tính toán thực tế hoàn toàn tách biệt nhau:

1. **Run Coordinator:** Lớp này được kích hoạt bởi tiến trình Webserver khi một tác vụ được khởi chạy từ giao diện UI. Chức năng chính của Run Coordinator là chuyển giao thông tin tác vụ đến Daemon thông qua một cơ chế hàng đợi.

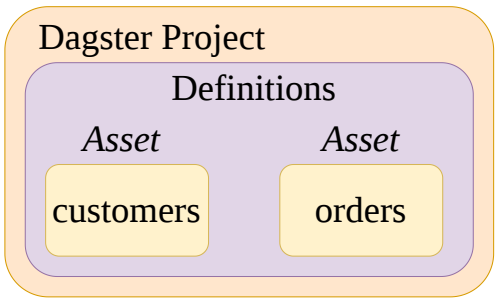
2. **Run Launcher:** Khi tiến trình Daemon nhận được một tác vụ từ hàng đợi, lớp Run Launcher sẽ được gọi để khởi tạo một worker xử lý mới. Tùy thuộc vào thiết lập hạ tầng, đơn vị này có thể là một tiến trình cục bộ mới, một container Docker hoặc một Kubernetes pod.

3. **Run Worker:** Đây là tiến trình trực tiếp nhận trách nhiệm duyệt qua đồ thị tính toán của tác vụ. Nhiệm vụ của tiến trình này là phối hợp với thành phần Executor để kích hoạt tuần tự hoặc song song các tác vụ trong quy trình.

4. **Executor:** Lớp Executor được Run Worker gọi trực tiếp nhằm thực thi các đoạn mã thuật toán do người dùng định nghĩa. Môi trường thực thi của Executor phụ thuộc vào cấu hình tài nguyên của tác vụ.

III. Các khái niệm cốt lõi trong Dagster

III.1. Software-Defined Assets (SDAs)



Hình 5: Một Dagster project đơn giản với hai asset `customers` và `orders` được đăng ký trong đối tượng `Definitions`.

Software-Defined Asset là nền tảng triết lý của Dagster, đánh dấu sự dịch chuyển từ mô hình hướng tác vụ (như Apache Airflow) sang mô hình hướng dữ liệu. Mỗi asset đại diện cho một đối tượng dữ liệu cụ thể, chẳng hạn một bảng trong cơ sở dữ liệu, một tệp tin hoặc một mô hình học máy, đi kèm với đoạn mã dùng để tạo ra nó. Như minh họa trong Hình 5, các asset như `customers` và `orders` được khai báo trong `Definitions` của project. Nhờ cách tổ chức này, ta có thể theo dõi trực tiếp nguồn gốc, mối quan hệ phụ thuộc và trạng thái cập nhật của dữ liệu, thay vì chỉ quan tâm đến việc tác vụ nào đã chạy.

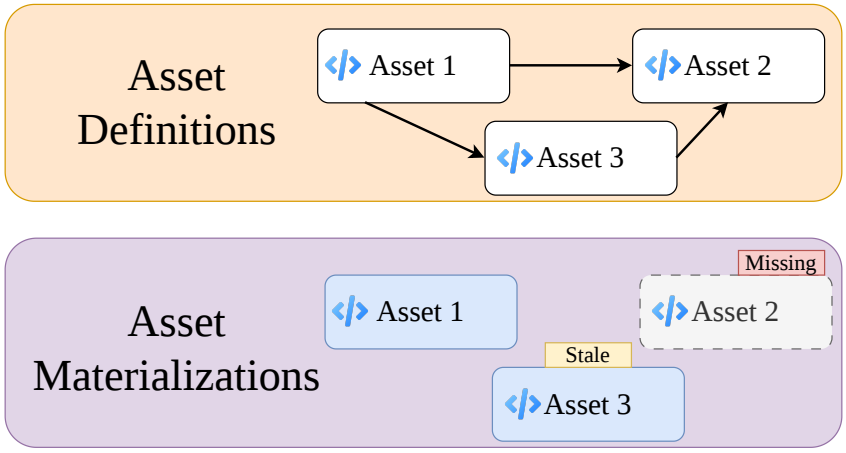
Kỹ thuật khai báo dữ liệu bắt đầu bằng việc định nghĩa các assets bằng mã nguồn, mô tả không chỉ bản chất của các assets đó mà còn cả cách thức chúng được tạo ra và vị trí của chúng trong tổng thể nền tảng. Mỗi định nghĩa assets trong Dagster bao gồm:

- Một asset key đóng vai trò là định danh toàn cục.

- Một op là hàm xử lý dùng để tạo ra asset.
- Một tập hợp các asset keys dùng để xác định các dependencies.

Việc kết hợp những thành phần này tạo ra một mô hình vững chắc để xây dựng các ứng dụng dữ liệu. Bằng cách theo dõi các yếu tố phụ thuộc ngay tại mỗi node, Dagster có thể tự động suy luận ra asset graph, từ đó loại bỏ hoàn toàn nhu cầu định nghĩa thủ công các sơ đồ luồng (DAG) cứng nhắc, vốn rất dễ bị lỗi thời hoặc không khớp với các luồng phụ thuộc thực tế.

Ngoài ra, khi làm việc với asset, cần phân biệt hai khái niệm: **asset definition** và **asset materialization**. Asset definition là phần mã nguồn mô tả cách asset được tạo ra và phụ thuộc vào những asset nào. Trong khi đó, asset materialization là lần thực thi thực sự đoạn mã đó để tạo ra hoặc cập nhật dữ liệu. Dagster ghi lại mỗi lần materialize, nhờ vậy có thể biết asset nào chưa từng được tạo, asset nào đã lỗi thời so với dữ liệu đầu vào (xem Hình 6).



Hình 6: Asset definition mô tả các asset và quan hệ phụ thuộc, còn asset materialization phản ánh trạng thái dữ liệu thực tế của từng asset.

Một Software-defined asset là một hàm Python được gắn decorator `@asset`, dùng để mô tả cách thức tạo ra một phần dữ liệu cụ thể. Hàm này đóng vai trò là phần logic xử lý, còn decorator có nhiệm vụ đăng ký nó thành một asset trong danh mục của Dagster.

Ví dụ về khai báo quan hệ phụ thuộc giữa các asset

```python
from dagster import asset
import pandas as pd

@asset
def raw_orders():
    return pd.read_csv("s3://my-bucket/orders/raw.csv")

@asset
def cleaned_orders(raw_orders):
    df = raw_orders.dropna().copy()
    df["order_date"] = pd.to_datetime(df["order_date"])
    return df
```

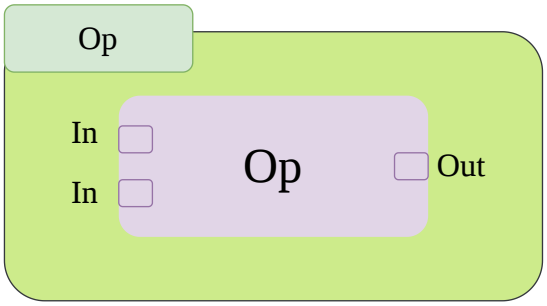
```python

@asset
def monthly_revenue(cleaned_orders):
    return cleaned_orders.groupby(
        cleaned_orders["order_date"].dt.month
    )["amount"].sum()
```

Ở code trên, ba assets có các mối phụ thuộc rõ ràng. Cụ thể, Dagster tự động đọc các phần khai báo hàm (*function signature*) và xây dựng đồ thị phụ thuộc. Ngoài ra, `monthly_revenue` phụ thuộc vào `cleaned_orders`, và tiếp đó `cleaned_orders` lại phụ thuộc vào `raw_orders`. Hoàn toàn không cần thao tác liên kết DAG thủ công.

III.2. Ops, Asset jobs, I/O Manager, Partitions

III.2.1. Ops



Hình 7: Cấu trúc của một op: nhận dữ liệu qua các đầu vào (In), thực hiện tính toán và trả kết quả qua đầu ra (Out).

Trong Dagster, op là đơn vị tính toán cơ bản nhất. Mỗi op là một hàm Python được đánh dấu bằng decorator `@dg.op`, và hàm này được gọi là compute function. Thông thường, một op nên đảm nhận một tác vụ tương đối đơn giản và có phạm vi rõ ràng, chẳng hạn như truy vấn cơ sở dữ liệu, gọi một API rồi lưu kết quả vào kho dữ liệu, khởi chạy một Spark job trên cụm từ xa, hoặc gửi thông báo qua email hay Slack.

Bên dưới mỗi asset thực chất là một op, đơn vị tính toán cơ bản của Dagster. Có thể hiểu op là một hàm Python thực hiện một công việc cụ thể, còn asset là op được gắn thêm thông tin về dữ liệu mà nó tạo ra, chẳng hạn tên của asset và các asset mà nó phụ thuộc.

Ví dụ về định nghĩa op và ghép các op thành job

```python
import requests
import dagster as dg

class FetchConfig(dg.Config):
    api_endpoint: str

@dg.op
def fetch_articles(context: dg.OpExecutionContext,
                   config: FetchConfig) -> list[dict]:
    context.log.info(f"Run ID: {context.run_id}")
    return requests.get(f"{config.api_endpoint}/articles").json()
```

```python

@dg.op
def count_articles(articles: list[dict]) -> int:
    return len(articles)

@dg.job
def article_op_job():
    count_articles(fetch_articles())
```

Trong ví dụ trên, op `fetch_articles` lấy danh sách bài viết từ API với địa chỉ được truyền qua cấu hình `FetchConfig`, còn op `count_articles` đếm số bài viết nhận được. Hàm `article_op_job` ghép hai op lại bằng cách nối đầu ra của `fetch_articles` vào đầu vào của `count_articles`, đúng với cơ chế đầu vào và đầu ra được mô tả trong Hình 7. Khác với asset, thứ tự và luồng dữ liệu giữa các op phải được khai báo rõ ràng trong job (xem thêm tại tài liệu Op jobs của Dagster).

III.2.2. Asset jobs

Asset job là job nhắm tới một tập con asset được chọn (asset selection). Asset job được tạo bằng hàm `define_asset_job`, và phạm vi thực thi của nó được xác định dựa trên các asset mục tiêu và quan hệ phụ thuộc giữa chúng. Một job có thể nhắm tới một hoặc nhiều asset, và nhiều job khác nhau hoàn toàn có thể chồng lấn lên cùng một tập hợp asset. Asset job có thể được kích hoạt theo ba cách: thủ công từ giao diện Dagster, định kỳ thông qua schedule, hoặc khi có thay đổi từ bên ngoài thông qua sensor.

Định nghĩa asset và các asset job

```python
import dagster as dg

@dg.asset
def raw_articles() -> None:
    ...

@dg.asset(deps=[raw_articles])
def cleaned_articles() -> None:
    ...

all_assets_job = dg.define_asset_job(name="all_assets_job")

raw_only_job = dg.define_asset_job(
    name="raw_only_job", selection="raw_articles"
)
```

III.2.3. I/O Manager

Trong nhiều pipeline, một asset thường tuân theo mẫu ba bước: đọc dữ liệu từ kho lưu trữ vào bộ nhớ, biến đổi dữ liệu trong bộ nhớ, rồi ghi kết quả trở lại kho lưu trữ. I/O manager được thiết kế để tách phần mã xử lý dữ liệu khỏi phần mã đọc và ghi dữ liệu. Nhờ đó, mã lặp lại ở đầu và cuối mỗi asset được loại bỏ và việc thay đổi nơi lưu trữ trở nên đơn giản hơn.

Một trong những ưu điểm của cách tiếp cận này là khả năng hoán đổi kho lưu trữ. Khi chuyển từ môi trường phát triển sang môi trường production (ví dụ từ DuckDB sang Snowflake), chỉ cần thay đổi cài đặt I/O manager trong phần khai báo resource, còn định nghĩa asset, vốn chỉ chứa logic biến đổi, được giữ nguyên. Dagster cung cấp sẵn nhiều I/O manager, được tóm tắt trong Bảng 1.

Bảng 1: Một số I/O manager dựng sẵn trong Dagster

I/O manager	Chức năng
`FilesystemIOManager`	Mặc định; lưu đầu ra dạng tệp pickle trên hệ thống tệp cục bộ
`InMemoryIOManager`	Lưu đầu ra trong bộ nhớ, chủ yếu phục vụ kiểm thử đơn vị
`S3PickleIOManager`, `GCSPickleIOManager`	Lưu tệp pickle trên AWS S3 hoặc Google Cloud Storage
`DuckDBPandasIOManager`, `SnowflakePandasIOManager`, `BigQueryPandasIOManager`	Lưu Pandas DataFrame vào DuckDB, Snowflake hoặc BigQuery

Asset sử dụng I/O manager để đọc và ghi DataFrame

```python
import pandas as pd
import dagster as dg
from dagster_duckdb_pandas import DuckDBPandasIOManager

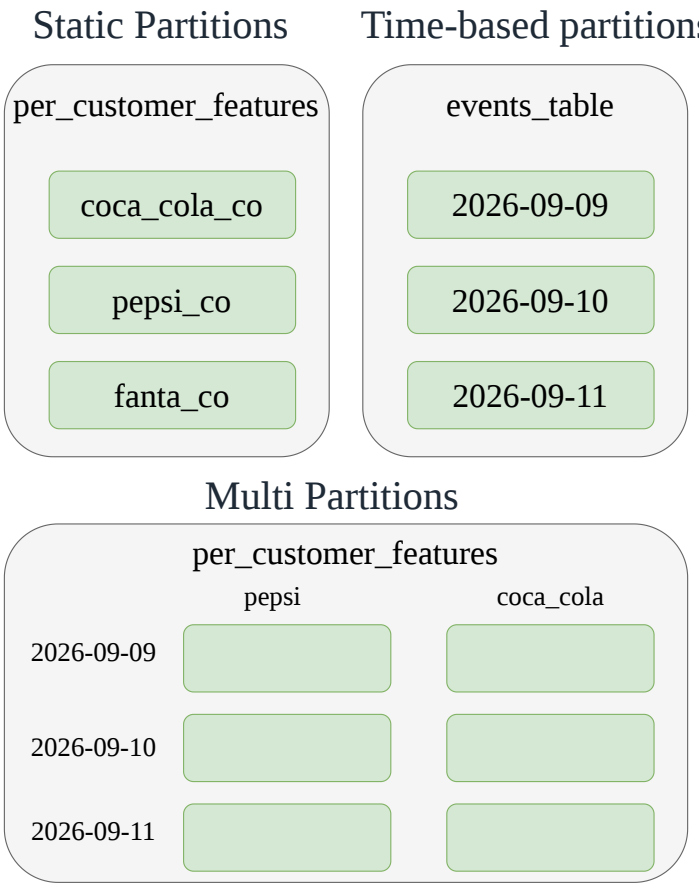
@dg.asset
def raw_articles() -> pd.DataFrame:
    return pd.read_csv("data/articles.csv")

@dg.asset
def cleaned_articles(raw_articles: pd.DataFrame) -> pd.DataFrame:
    return raw_articles.dropna(subset=["title"])

defs = dg.Definitions(
    assets=[raw_articles, cleaned_articles],
    resources={
        "io_manager": DuckDBPandasIOManager(
            database="news.duckdb", schema="public"
        )
    },
)
```

III.2.4. Partitions

Phân vùng là kỹ thuật chia một asset thành các phần logic độc lập, thường theo thời gian, nhằm quản lý các tập dữ liệu lớn, cải thiện hiệu năng pipeline và hỗ trợ xử lý tăng dần. Dagster theo dõi trạng thái materialize của từng partition, nhờ đó người dùng có thể thực hiện backfill cho một khoảng thời gian cụ thể mà không cần chạy lại toàn bộ lịch sử dữ liệu.



Hình 8: Các kiểu phân vùng được Dagster hỗ trợ, tương ứng với những cách chia nhỏ dữ liệu khác nhau tùy theo nhu cầu xử lý.

Có một số cách để phân vùng dữ liệu trong Dagster:

- **Time-based partitioning**: để xử lý dữ liệu theo các khoảng thời gian cụ thể.
- **Static partitioning**: để chia nhỏ dữ liệu dựa trên các danh mục được định nghĩa trước.
- **Two-dimensional partitioning**: để phân vùng dữ liệu dọc theo hai trục khác nhau đồng thời.
- **Dynamic partitioning**: để tạo các phân vùng dựa trên thông tin tại thời điểm chạy.

Để khai báo phân vùng cho một asset, ta truyền một đối tượng định nghĩa phân vùng vào tham số `partitions_def` của decorator `@asset`. Trong ví dụ dưới đây, hai asset `events` và `logins`

được phân vùng theo giờ, còn `logins_cleaned` được phân vùng theo ngày. Dù có mức phân vùng khác nhau, các asset này vẫn phụ thuộc lẫn nhau: Dagster tự ánh xạ mỗi phân vùng ngày của `logins_cleaned` với 24 phân vùng giờ tương ứng của `events` và `logins`.

Asset phân vùng theo giờ và theo ngày

```python
from dagster import DailyPartitionsDefinition, HourlyPartitionsDefinition, asset

@asset(partitions_def=HourlyPartitionsDefinition(start_date="2026-01-01-00:00"))
def events():
    ...

@asset(partitions_def=HourlyPartitionsDefinition(start_date="2026-01-01-00:00"))
def logins(events):
    ...

@asset(partitions_def=DailyPartitionsDefinition(start_date="2026-01-01"))
def logins_cleaned(events, logins):
    ...
```

Lưu ý

Khuyến nghị nên giới hạn số lượng phân vùng cho mỗi asset ở mức 100.000 hoặc ít hơn. Các asset có số lượng phân vùng vượt quá giới hạn này có khả năng sẽ làm giao diện người dùng tải chậm hơn.

III.3. Schedules và Sensors

Dagster cung cấp hai cơ chế tự động hóa bổ trợ cho nhau: schedule kích hoạt theo thời gian và sensor kích hoạt theo sự kiện. Cả hai đều được định nghĩa bằng Python cùng với mã nguồn pipeline và được vận hành bởi tiến trình dagster-daemon, trong đó scheduler daemon tạo run từ các schedule đang hoạt động, còn sensor daemon tạo run từ các sensor đang hoạt động.

III.3.1. Schedules

Schedule cho phép thực thi job một cách tự động theo chu kỳ xác định, từ các tần suất phổ biến như hằng giờ, hằng ngày, hằng tuần cho đến những mẫu phức tạp hơn được mô tả bằng biểu thức cron. Một schedule cơ bản được tạo bằng lớp `ScheduleDefinition`, gồm hai thành phần chính: đối tượng cần thực thi (một job, hoặc trực tiếp một danh sách asset thông qua tham số `target`) và biểu thức `cron_schedule`.

Một số điểm cần lưu ý khi thiết kế schedule:

- **Múi giờ**: nếu không chỉ định, schedule mặc định chạy theo giờ UTC. Để chạy theo giờ địa phương, cần thiết lập tham số `execution_timezone`, ví dụ `"Asia/Ho_Chi_Minh"`.
- **Schedule từ job phân vùng**: với job có partition, hàm `build_schedule_from_partitioned_job` tự động sinh schedule có tần suất khớp với định nghĩa partition, và mỗi lần kích hoạt sẽ materialize partition tương ứng.

Đoạn code dưới đây minh họa cách tạo schedule nêu trên. Trước tiên, asset `article_report` được gom vào job `report_job` thông qua hàm `define_asset_job`. Schedule `report_schedule` sau đó kích hoạt job này với biểu thức cron `"0 1 * * *"`, nghĩa là chạy vào phút 0 của 1 giờ sáng mỗi ngày, và nhờ tham số `execution_timezone` nên thời điểm này được tính theo giờ Việt Nam thay vì UTC.

Schedule theo cron_schedule

```python
import dagster as dg

@dg.asset
def article_report() -> None:
    pass

report_job = dg.define_asset_job("report_job", selection=[article_report])

report_schedule = dg.ScheduleDefinition(
    job=report_job,
    cron_schedule="0 1 * * *",
    execution_timezone="Asia/Ho_Chi_Minh",
)

defs = dg.Definitions(
    assets=[article_report],
    jobs=[report_job],
    schedules=[report_schedule]
)
```

III.3.2. Sensors

Khác với schedule, sensor cho phép pipeline phản ứng với các sự kiện xảy ra bên trong Dagster hoặc ở các hệ thống bên ngoài. Sensor hoạt động theo cơ chế thăm dò (polling): định kỳ kiểm tra điều kiện, sau đó hoặc thực hiện hành động, hoặc ghi lại lý do bỏ qua. Các sự kiện điển hình bao gồm một run hoàn thành hoặc thất bại, một asset vừa được materialize, một tệp mới xuất hiện trong bucket S3, hoặc một hệ thống bên ngoài ngừng hoạt động. Hành động tương ứng có thể là khởi chạy run, gửi tin nhắn Slack hoặc chèn bản ghi vào cơ sở dữ liệu.

Sensor được định nghĩa bằng decorator `@dg.sensor`, với các cơ chế chính sau:

- **Kết quả đánh giá**: mỗi lần đánh giá, sensor trả về `RunRequest` để yêu cầu khởi chạy run, hoặc `SkipReason` để giải thích vì sao không có run nào được tạo; lý do này hiển thị trên giao diện Dagster.
- **Chu kỳ đánh giá**: tham số `minimum_interval_seconds` quy định khoảng thời gian tối thiểu giữa hai lần đánh giá, chứ không phải tần suất chính xác. Nếu hàm đánh giá chạy lâu hơn khoảng thời gian này, lần đánh giá kế tiếp sẽ bị lùi lại tương ứng.
- **Chống trùng lặp bằng `run_key`**: với cùng một sensor, mỗi `run_key` duy nhất chỉ tạo ra một run; các yêu cầu có key đã dùng sẽ bị bỏ qua.

- **Cursor**: với luồng sự kiện khối lượng lớn, sensor có thể lưu trạng thái tùy biến qua `context.cursor` và `context.update_cursor()`, chẳng hạn thời điểm sửa đổi gần nhất của tệp đã xử lý. Cần lưu ý rằng nếu dùng đồng thời cursor và `run_key`, việc đặt lại cursor sẽ không tạo run mới cho các tệp không thay đổi, vì run key của chúng đã tồn tại.

Bên cạnh sensor tổng quát, Dagster còn cung cấp các biến thể chuyên biệt như *asset sensor* (kích hoạt khi một asset cụ thể được materialize) và *run status sensor* (kích hoạt theo trạng thái của run). Ngoài ra, thay vì để sensor thăm dò, hệ thống bên ngoài cũng có thể chủ động đẩy sự kiện vào Dagster thông qua GraphQL API.

Sensor phát hiện tệp mới sử dụng cursor

```python
import os
import dagster as dg

INBOX = "data/inbox"

@dg.sensor(
    job=report_job,
    minimum_interval_seconds=30,
    default_status=dg.DefaultSensorStatus.RUNNING,
)
def new_file_sensor(context: dg.SensorEvaluationContext):
    last_mtime = float(context.cursor) if context.cursor else 0.0
    max_mtime, found = last_mtime, False

    for name in os.listdir(INBOX):
        mtime = os.stat(os.path.join(INBOX, name)).st_mtime
        if mtime > last_mtime:
            found = True
            yield dg.RunRequest(run_key=f"{name}:{mtime}")
            max_mtime = max(max_mtime, mtime)

    context.update_cursor(str(max_mtime))
    if not found:
        yield dg.SkipReason("No new files found")
```

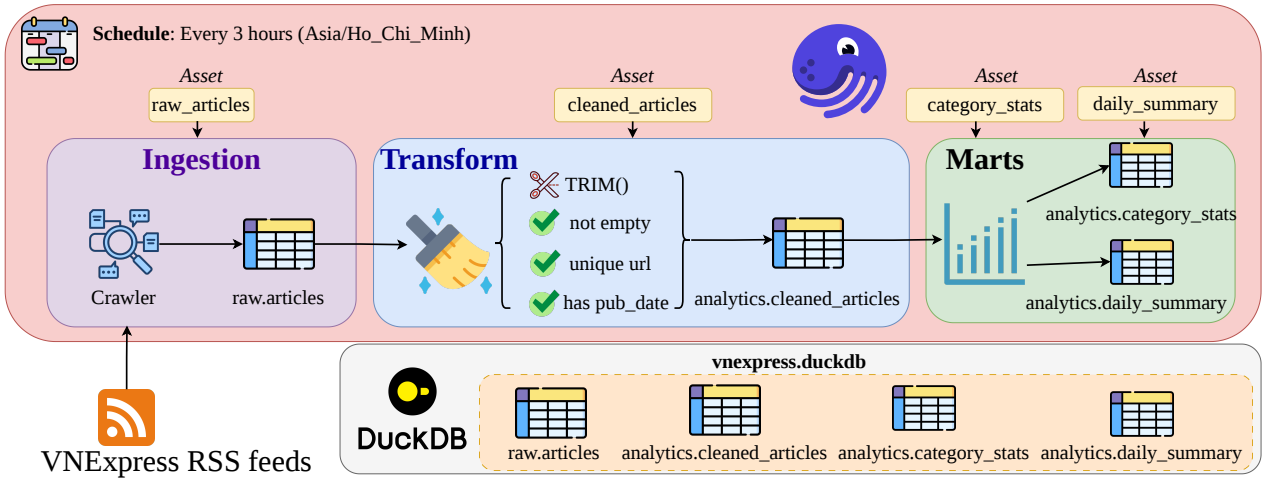
Đoạn code trên minh họa một sensor theo dõi thư mục `data/inbox` và kích hoạt `report_job` mỗi khi có tệp mới. Cứ tối thiểu 30 giây, sensor duyệt qua các tệp trong thư mục và so sánh thời điểm sửa đổi của từng tệp với giá trị đã lưu trong cursor. Với mỗi tệp mới hơn, sensor trả về một `RunRequest` có `run_key` gồm tên tệp và thời điểm sửa đổi, để cùng một tệp không bị xử lý hai lần. Sau đó cursor được cập nhật bằng thời điểm sửa đổi lớn nhất, và nếu không có tệp mới, sensor trả về `SkipReason`. Tham số `default_status` giúp sensor tự động bật khi được triển khai.

IV. Thực hành

IV.1. Kiến trúc tổng quan

Trong bài thực hành này, chúng ta sẽ vận dụng Dagster để xây dựng hoàn chỉnh một pipeline dữ liệu cho bài toán thu thập tin tức từ VNExpress, đi từ khâu lấy dữ liệu thô đến các bảng thống kê phục vụ phân tích.

Pipeline gồm bốn *asset* chính, chạy nối tiếp nhau và được nhóm theo ba giai đoạn: Ingestion, Transform, và Marts. Hình 9 minh họa toàn bộ luồng xử lý, từ nguồn RSS cho đến các bảng dữ liệu cuối cùng trong file DuckDB duy nhất `vnexpress.duckdb`.



Hình 9: Kiến trúc pipeline VNExpress: bốn asset chạy nối tiếp qua ba giai đoạn ingestion, transform và marts, với toàn bộ dữ liệu lưu trong tệp `vnexpress.duckdb`.

IV.2. Vai trò từng thành phần

Bảng 2: Bốn asset chính trong pipeline và vai trò của chúng.

Asset	Nhóm	Bảng đầu ra	Vai trò
`raw_articles`	ingestion	`raw.articles`	Cào link bài viết từ RSS theo từng chuyên mục, sau đó cào nội dung đầy đủ của từng bài (tiêu đề, mô tả, nội dung, ngày đăng) bằng nhiều luồng song song.
`cleaned_articles`	transform	`analytics.cleaned_articles`	Chuẩn hóa chuỗi (`TRIM`), loại bài không có tiêu đề, khử trùng lặp theo URL (giữ bản ghi cào mới nhất).

Tiếp tục ở trang sau

Bảng 2: Bốn asset chính trong pipeline và vai trò của chúng.

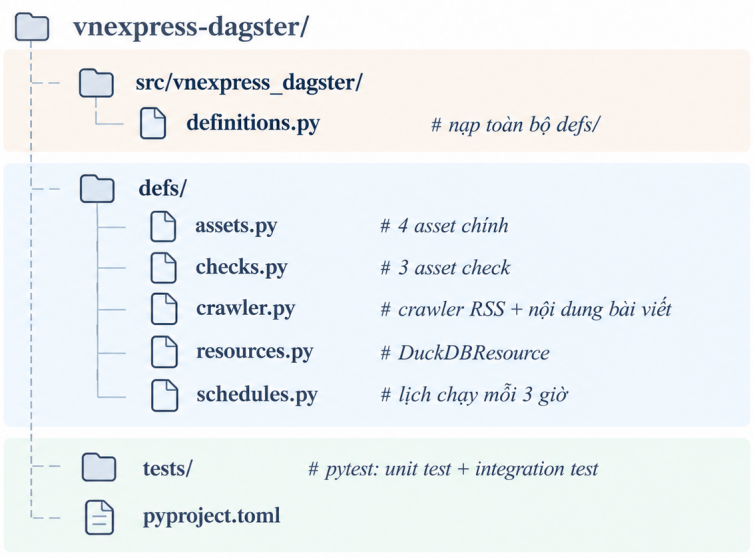
Asset	Nhóm	Bảng đầu ra	Vai trò
`category_stats`	marts	`analytics.category_stats`	Số lượng bài, thời điểm đăng mới nhất, độ dài nội dung trung bình.
`daily_summary`	marts	`analytics.daily_summary`	Số lượng bài theo từng ngày đăng (giờ Việt Nam) và chuyên mục.

Ngoài bốn asset trên, `cleaned_articles` còn được gắn ba *asset check* để đảm bảo chất lượng dữ liệu trước khi các mart phía sau chạy:

- `cleaned_articles_not_empty`: Nếu bảng rỗng, các asset phía sau sẽ không chạy.
- `cleaned_articles_unique_url`: Kiểm tra không còn URL trùng lặp sau bước làm sạch.
- `cleaned_articles_has_pub_date`: Cảnh báo nhưng không chặn pipeline nếu có bài thiếu ngày đăng hợp lệ.

Toàn bộ pipeline được tự động hóa bằng một schedule chạy mỗi 3 giờ (`0 */3 * * *`, múi giờ `Asia/Ho_Chi_Minh`); lịch này ở trạng thái tắt theo mặc định để tránh việc pipeline tự động cào dữ liệu ngay khi mới chạy thử `dg dev`.

IV.3. Cấu trúc thư mục



Hình 10: Cấu trúc thư mục dự án

Cấu trúc dự án tuân theo quy ước chuẩn của dg CLI (Dagster), tách phần logic cào dữ liệu khỏi phần định nghĩa asset. `definitions.py` nạp toàn bộ nội dung trong thư mục `defs/`, nơi khai báo 4 asset (`assets.py`), 3 asset check (`checks.py`), resource kết nối DuckDB (`resources.py`), và lịch chạy mỗi 3 giờ (`schedules.py`); logic cào RSS và nội dung bài viết nằm riêng trong `crawler.py`. Thư mục `tests/` chứa cả unit test cho crawler và integration test cho toàn bộ pipeline.

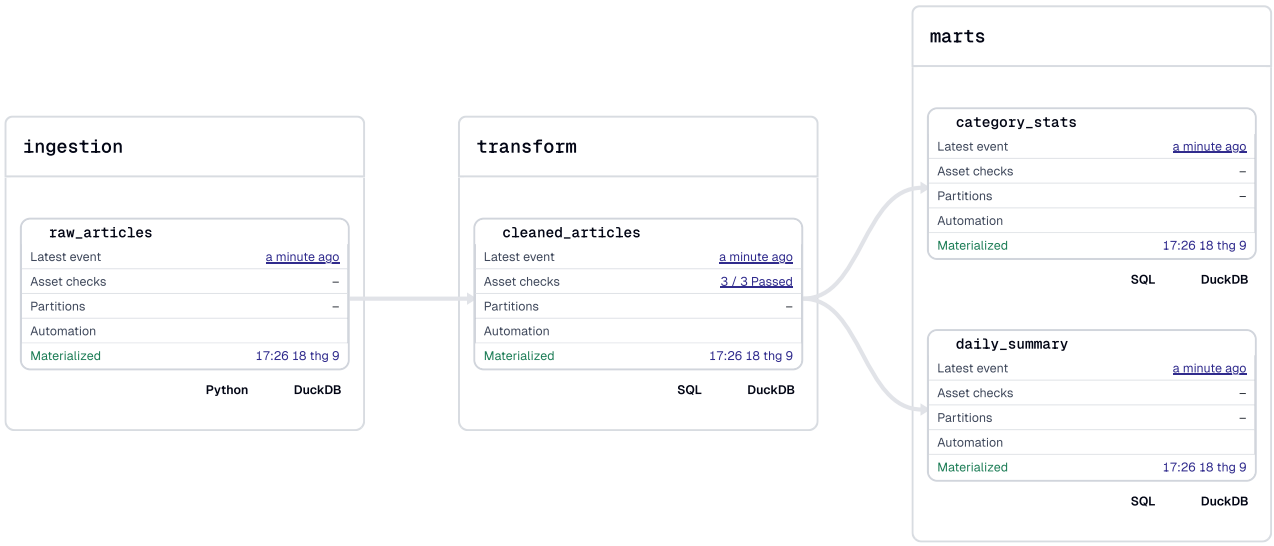
V. Triển khai

V.1. Cài đặt và chạy

PowerShell Commands

```
uv sync # Cai dat dependencies va tao virtual environment
copy .env.example .env # Sao chep file cau hinh mau
.venv\Scripts\activate # Kich hoat virtual environment
dg dev # Khoi chay Dagster
```

Sau khi `dg dev` khởi động, mở trình duyệt tại http://localhost:3000, vào tab **Lineage** và bấm **Materialize all** để chạy toàn bộ pipeline một lần bằng dữ liệu cào trực tiếp từ VNExpress. Khi quá trình hoàn tất, asset graph của dự án sẽ hiển thị như trong Hình 11.



Hình 11: Asset graph của dự án sau khi materialize thành công, với các asset được nhóm theo ba tầng ingestion, transform và marts.

Giao diện cho thấy quan hệ phụ thuộc giữa các asset cùng thời điểm materialize gần nhất của từng asset, qua đó có thể theo dõi trực quan trạng thái của toàn bộ pipeline.

V.2. Chạy với cấu hình tùy chỉnh

Asset `raw_articles` nhận run config, có thể chỉnh trực tiếp trên launchpad của UI hoặc qua CLI:

Bash Commands

```
dg launch --assets=* --config-json '{\"ops\": {\"raw_articles\":
{\"config\": {\"categories\": [\"the-thao\", \"giai-tri\"],
\"max_articles_per_category\": 5}}}}'
```

Tham số `max_articles_per_category` (mặc định 10) giới hạn số trang bài viết được cào mỗi lần chạy, vì mỗi bài là một request HTTP riêng. Nếu quá trình cào không lấy được dữ liệu (ví

dụ không có kết nối mạng), asset ghi log cảnh báo và tiếp tục với 0 dòng; check `cleaned_articles_not_empty` khi đó sẽ chặn các mart phía sau chạy trên dữ liệu rỗng.

V.3. Kiểm tra dữ liệu sau khi chạy

PowerShell Commands

```
python -c "import duckdb; c = duckdb.connect('data/vnexpress.duckdb',
read_only=True); print(c.sql('SELECT * FROM analytics.category_stats'))"
```

V.4. Kiểm thử

PowerShell Commands

```
uv run pytest # unit test crawler + integration test toan bo pipeline
uv run dg check defs # kiem tra cau hinh Dagster nap dung
```

`tests/test_pipeline.py` chạy toàn bộ 4 asset và 3 asset check trong một lệnh `dg.materialize()` duy nhất, trên một file DuckDB tạm thời. Đây cũng là điểm khác biệt đáng chú ý so với Airflow: vì asset trong Dagster chỉ là hàm Python thuần với resource được inject qua tham số, toàn bộ đồ thị phụ thuộc có thể chạy ngay trong một hàm `pytest` bình thường và không cần scheduler, webserver hay metadata database đang chạy như khi kiểm thử một DAG Airflow ở mức thực thi đầy đủ (thường phải dùng `airflow dags test` hoặc dựng một Airflow instance cục bộ). Nhờ đó bài test chạy trong vài giây, lặp lại được, và không phụ thuộc hạ tầng bên ngoài.

VI. Tài liệu tham khảo

- https://docs.dagster.io/deployment/oss/oss-deployment-architecture
- https://dagster.io/blog/dagster-code-locations
- https://atlan.com/dagster-data-orchestration/
- https://docs.dagster.io/guides
- https://dagster.io/blog/software-defined-assets

Source code: https://github.com/undertanker86/vnexpress-dagster-orches

- Hết -